PERTEMUAN 3 · Praktik terbimbing

# Analisis Leksikal: Token, Lexeme, Pattern, dan Ekspresi Reguler

**Universitas Bale Bandung**

Fakultas Teknologi Informasi · Program Studi Teknik Informatika

Mata Kuliah Teknik Kompilasi (MWCA017) · Semester Ganjil · 3 SKS

Dosen: Cecep Suwanda, S.Si., M.Kom.

# Capaian Pertemuan

**CPMK induk:** CPMK 02 — Memahami konsep Otomata Hingga (*Finite Automata*).

## Sub-CPMK yang dinilai pada pertemuan ini

**Sub-CPMK 2.1**: Membuat tabel ekspresi reguler untuk setiap kategori token subset bahasa C (kata kunci, *identifier*, bilangan, operator, dan pemisah).

## Luaran yang diperiksa

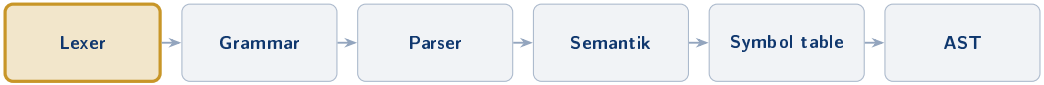
Tabel ekspresi reguler yang lengkap, contoh lexeme untuk setiap kategori, dan spesifikasi token Subset-C beserta kasus ujinya.

# Agenda Pertemuan (150 menit)

| Menit | Kegiatan |
|---|---|
| 30 | Ceramah: peran lexer, token/lexeme/pattern, ekspresi reguler |
| 20 | Demonstrasi: menokenisasi program secara manual |
| 60 | Praktik terbimbing: menyusun tabel ekspresi reguler |
| 30 | Perancangan 30 kasus uji dan format keluaran |
| 10 | Pemeriksaan bersama, kesalahan umum, dan penutup |

Porsi praktik 120 menit lebih besar daripada ceramah 30 menit. Setiap mahasiswa mengumpulkan tabel dan kasus uji pada akhir pertemuan.

# Posisi Pertemuan pada Rantai Proyek

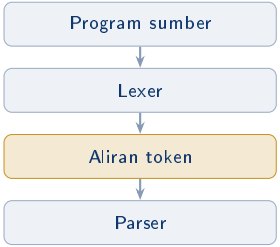



- Pertemuan ini membangun simpul **lexer**: dari spesifikasi token menuju mesin pengenal.
- Keluaran pertemuan menjadi masukan langsung untuk NFA/DFA pada Pertemuan 4 dan implementasi lexer pada Pertemuan 5.
- Setiap token yang dirancang hari ini harus dapat dikenali oleh automata yang dibangun berikutnya.

# Tugas Scanner

- Lexer (*scanner*) adalah fase pertama kompilator.
- Membaca program sumber karakter demi karakter.
- Membuang spasi, baris baru, dan komentar.
- Mengelompokkan karakter menjadi token, lalu menyerahkan token ke parser.
- Mencatat identifier ke tabel simbol bila diperlukan.

Antarmuka *pull model*: parser memanggil `next_token()` untuk setiap token berikutnya.




Lexer tidak memahami makna program; itu tugas analisis semantik.

# Kontrak Lexer

- Mengembalikan token berikutnya, atau `TOK_EOF` pada akhir masukan.
- Mengisi atribut token: jenis, lexeme, baris, kolom, dan nilai bila ada.
- Melewati spasi, tab, baris baru, dan komentar.
- Menghasilkan `TOK_ERROR` untuk karakter ilegal tanpa menghentikan seluruh proses.
- Tidak mengubah program sumber.

**Mengapa kontrak ini penting**

Parser dan pengujian bergantung pada perilaku yang konsisten. Jika lexer berhenti pada kesalahan pertama, parser tidak dapat melakukan pemulihan.

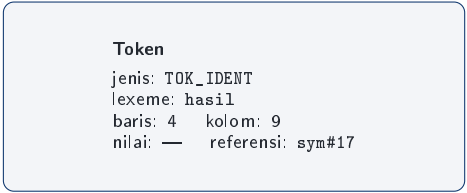
# Besaran Leksik: Token, Lexeme, dan Pattern

| Token | Lexeme | Pattern (informal) |
|---|---|---|
| IDENT | `hasil`, `_x1` | huruf atau garis bawah, diikuti huruf/angka/garis bawah |
| INT_LIT | `0`, `42` | satu atau lebih digit |
| FLOAT_LIT | `3.14` | digit, titik, digit |
| PLUS | `+` | karakter + |
| LE | `<=` | < diikuti = |
| STRING_LIT | `"halo"` | kutip ganda, isi, kutip ganda |

Token adalah kategori; lexeme adalah string konkret; pattern adalah aturan yang mendefinisikan bentuk lexeme. Satu token dapat memiliki banyak lexeme.

# Atribut Token

- **lexeme**: string asli;
- **baris** dan **kolom**: lokasi untuk diagnostik;
- **nilai**: bilangan, karakter, atau referensi;
- **indeks simbol**: rujukan ke tabel simbol bila diperlukan.




Anatomi token dan atributnya

# Contoh: Menokenisasi Program secara Manual

```
int x = 10;
x = x + 1;
```

Listing 1: Program uji tokenisasi

Spasi tidak menghasilkan token; setiap token memiliki lokasi awal.

| Baris:Kolom | Token | Lexeme |
|---|---|---|
| 1:1 | TOK_INT | int |
| 1:5 | TOK_IDENT | x |
| 1:7 | TOK_ASSIGN | = |
| 1:9 | TOK_INT_LIT | 10 |
| 1:11 | TOK_SEMI | ; |
| 2:1 | TOK_IDENT | x |
| 2:3 | TOK_ASSIGN | = |
| 2:5 | TOK_IDENT | x |
| 2:7 | TOK_PLUS | + |
| 2:9 | TOK_INT_LIT | 1 |
| 2:10 | TOK_SEMI | ; |
| 3:1 | TOK_EOF | — |

# Ekspresi Reguler: Definisi dan Prioritas

**Definisi rekursif**

$\varnothing$, $\varepsilon$, dan setiap $a \in \Sigma$ adalah ekspresi reguler. Jika $r$ dan $s$ ekspresi reguler, maka $r \mid s$, $rs$, dan $r^*$ juga ekspresi reguler.

- Gabungan $r \mid s$, konkatenasi $rs$, bintang Kleene $r^*$.
- $r^+$ adalah singkatan $rr^*$; $r?$ adalah $r \mid \varepsilon$.
- Prioritas dari tertinggi: bintang Kleene, konkatenasi, gabungan.
- $ab^* \mid c$ dibaca $(a(b^*)) \mid c$, bukan $(ab)^* \mid c$.

# Identitas Aljabar Ekspresi Reguler

- $r \mid s = s \mid r$
- $(rs)t = r(st)$
- $r \mid r = r$
- $r\varepsilon = \varepsilon r = r$
- $\varnothing \mid r = r$
- $\varnothing r = r\varnothing = \varnothing$
- $(r^*)^* = r^*$
- $(r \mid \varepsilon)^* = r^*$

**Kegunaan**

Alat seperti Flex memakai identitas ini untuk menyederhanakan otomata dan mempercepat pencocokan. Untuk spesifikasi token, kejelasan lebih penting daripada bentuk paling ringkas.

# Pola Umum Token

- identifier: `[A-Za-z_][A-Za-z0-9_]*`
- bilangan bulat: `[0-9]+`
- bilangan pecahan: `[0-9]+\.[0-9]+`
- string: `"([^"\\]|\\.)*"`
- komentar baris: `//[^\n]*`
- komentar blok: `/\*([^*]|\*[^/])*\*/`

**Catatan implementasi**

Notasi di atas memakai kelas karakter dan *escape*. Subset-C dibatasi pada ASCII agar spesifikasi dan pengujian lebih sederhana. Kasus batas seperti `/**/` dan komentar tidak ditutup harus diuji tersendiri.

# Maximal Munch dan Prioritas Aturan

**Aturan *maximal munch* (*longest match*)**

Ketika beberapa pattern cocok pada posisi yang sama, lexer memilih lexeme terpanjang.

- `a<=b`: pilih `LE`, bukan `LT` lalu `ASSIGN`.
- `a«b`: tidak ada « di Subset-C, sehingga lexer memilih `LT` dua kali.
- Jika panjang sama, aturan yang lebih awal dalam spesifikasi menang.
- Kata kunci ditempatkan sebelum identifier, atau identifier diperiksa ke tabel kata kunci setelah dicocokkan.

# Prioritas Aturan Token Subset-C

| Urutan | Kategori | Contoh |
|---|---|---|
| 1 | whitespace dan komentar | spasi, `//`, `/* */` |
| 2 | operator dua karakter | `<=`, `>=`, `==`, `!=`, `&&`, `\|\|` |
| 3 | operator satu karakter | `+`, `-`, `*`, `/`, `<`, `=` |
| 4 | literal float | `3.14`, `0.5` |
| 5 | literal integer | `0`, `42` |
| 6 | identifier dan kata kunci | `hasil`, `if` |
| 7 | literal char dan string | `'a'`, `"halo"` |
| 8 | karakter ilegal | `@`, `#` |

Urutan ini memastikan `<=` tidak dibaca sebagai `<` diikuti `=`, dan `3.14` tidak dibaca sebagai `3` lalu titik.

# Tabel Ekspresi Reguler Kategori Token (Artefak Sub-CPMK 2.1)

| Kategori | Pola (ekspresi reguler) | Contoh lexeme |
|---|---|---|
| Kata kunci | `if \| else \| while \| for \| int \| float \| char \| void \| return \| break \| continue \| print` | `if`, `while` |
| *Identifier* | `[A-Za-z_][A-Za-z0-9_]*` | `hasil`, `_x1` |
| Bilangan bulat | `[0-9]+` | `42` |
| Bilangan pecahan | `[0-9]+\.[0-9]+` | `3.14` |
| String | `"([^"\\]\|\\.)*"` | `"halo"` |
| Operator | `== \| != \| <= \| >= \| && \| \|\| \| [+-*/<>=!%]` | `<=`, `&&` |
| Pemisah | `( ) { } [ ] ; ,` | `(`, `;` |

Pola ini adalah spesifikasi, bukan kode C. Tabel yang lengkap menjadi artefak Sub-CPMK 2.1 dan dasar berkas `.l` pada Pertemuan 5.

# Contoh: Kata Kunci versus Identifier

**Tabel kata kunci**

`int`, `if`, `while`

**Masukan dan token**

`int x;` → `INT`, `IDENT(x)`, `SEMI`
`integer x;` → `IDENT(integer)`, `IDENT(x)`, `SEMI`

- `integer` tidak ada dalam tabel kata kunci, jadi tetap identifier.
- Lexer mencocokkan seluruh lexeme, bukan berdasarkan awalan.
- Strategi Subset-C: baca identifier, lalu periksa tabel kata kunci.
- Dengan cara ini, menambah kata kunci cukup mengubah tabel.

# Kesalahan Leksikal dan Pemulihannya

| Kasus | Perilaku lexer |
|---|---|
| Karakter ilegal, misalnya `@` | hasilkan `TOK_ERROR`, lanjutkan |
| String tidak ditutup | `TOK_ERROR` pada baris awal string |
| Char kosong atau terlalu panjang | `TOK_ERROR` |
| Komentar blok tidak ditutup | `TOK_ERROR` pada baris awal komentar |
| Float tanpa digit setelah titik | `TOK_ERROR`, misalnya `3.` |

**Pesan yang baik**

Sebut lokasi awal dan jenis masalah, misalnya `baris 1, kolom 10: digit diharapkan setelah titik`. Lexer tidak berhenti total; ia mencatat kesalahan lalu melanjutkan.

# Header `token.h`: Enum Jenis Token

```
8       TOK_EOF = 0,
9       TOK_ERROR,
10
11      /* Kata kunci */
12      TOK_INT,
13      TOK_FLOAT,
14      TOK_CHAR,
15      TOK_VOID,
16      TOK_IF,
17      TOK_ELSE,
18      TOK_WHILE,
19      TOK_FOR,
20      TOK_RETURN,
21      TOK_BREAK,
22      TOK_CONTINUE,
```

Listing 2: Bagian awal `token.h`: enum jenis token

- Enum memberi nama simbolik untuk setiap jenis token.
- `TOK_EOF` dan `TOK_ERROR` diletakkan paling awal.
- Kata kunci memiliki token sendiri.
- Struktur `Token` menyimpan lexeme, lokasi, dan nilai.

Nama token dipakai pada keluaran tes: `BARIS:KOLOM JENIS LEXEME`.

# Program Uji `lex_demo.sc`

```
// Berkas: examples/lex_demo.sc
// Program contoh untuk pengujian lexer.

// Identifier, literal, dan operator
int hitung(int a, int b) {
    int hasil = a + b * 2;
    float rasio = 3.14;
    char tanda = 'x';
    string pesan = "hasil = ";
    print(pesan);
    print(hasil);
    print(rasio);
```

Listing 3: Potongan program uji `lex_demo.sc`

- Memuat identifier, literal, dan operator.
- Baris berikutnya menambahkan `if`, `else`, `while`, `<=`, `&&`, dan `!=`.
- Berkas ini diuji pada lexer tangan dan Flex pada Pertemuan 5.
- Keluaran keduanya harus identik, kecuali perbedaan penanganan baris baru.

# Kegiatan Mahasiswa di Kelas: P1-A

## Spesifikasi token dan 30 kasus uji

1. Lengkapi tabel ekspresi reguler untuk setiap kategori token.
2. Tuliskan daftar kata kunci dan aturan escape char/string.
3. Buat 30 kasus uji: 10 normal, 10 batas, 10 kesalahan.
4. Untuk setiap kasus, tuliskan `.sc` dan `.expected` dengan format `BARIS:KOLOM JENIS LEXEME`.
5. Periksa bahwa urutan aturan memenuhi *maximal munch*.

Bekerja berpasangan; tukar tabel dengan pasangan lain untuk memeriksa konsistensi pattern dan contoh lexeme.

# Tugas dan Artefak

**Artefak Sub-CPMK 2.1**

Tabel ekspresi reguler untuk kata kunci, identifier, bilangan, operator, dan pemisah subset bahasa C, disertai contoh lexeme.

- Tabel lengkap untuk semua kategori token; tidak ada kategori tanpa pola.
- 30 kasus uji tersedia: 10 normal, 10 batas, 10 kesalahan.
- Format keluaran tes konsisten: `BARIS:KOLOM JENIS LEXEME`.
- Tidak ada aturan yang saling bertentangan; urutan aturan memenuhi *maximal munch*.

P1-A menjadi bagian Praktikum 1 (12%) yang dilaporkan pada Pertemuan 5.

# Rangkuman dan Jembatan

- Lexer membaca karakter, membuang whitespace/komentar, dan menghasilkan aliran token melalui *pull model*.
- Token adalah kategori, lexeme adalah string konkret, dan pattern adalah aturan pembentuknya.
- Ekspresi reguler dibangun dari gabungan, konkatenasi, dan bintang Kleene; prioritasnya bintang, konkatenasi, gabungan.
- *Maximal munch* dan urutan aturan menentukan token yang dipilih ketika beberapa pattern cocok.
- Tabel ekspresi reguler untuk semua kategori token adalah artefak Sub-CPMK 2.1.

**Pertemuan berikutnya**

Pertemuan 4 menghubungkan ekspresi reguler dengan mesin pengenal: NFA, DFA, konversi NFA ke DFA, dan minimalisasi DFA. Tabel pattern hari ini akan diubah menjadi automata yang diuji pada token.